

Student Course Management System

SCMS — Project Report

Course: Foundations of Software Development

Name: Mohammed Jameel Fouad

ID: 11216

Repository: github.com/iMohammedJameel/Student-Course-Management-System

Table of Contents

1. Introduction
2. Project Description
3. Project Beneficiaries
4. Detailed Analysis
5. Task Breakdown Structure
6. Time Plan — Gantt Chart
7. System Architecture and Design
8. Testing Scenarios and Results
9. End-User Guide
10. Conclusion
11. References

1. Introduction

This report documents the design, development, and testing of the Student Course Management System (SCMS) — a C++ console application submitted as an individual assignment for the Foundations of Software Development course. The system allows academic administrators to manage student records and course enrollments through a menu-driven interface.

The project applies core C++ concepts including Object-Oriented Programming (OOP), the Standard Template Library (STL), and algorithmic techniques. The codebase follows a two-file header structure:

- Student.h — contains the Person base class and the Student derived class
- CourseManager.h — contains the CourseManager class with all management logic
- main.cpp — entry point that instantiates CourseManager and calls run()

The GitHub repository for this project is available at:

<https://github.com/iMohammedJameel/Student-Course-Management-System>

2. Project Description

The SCMS is an eight-option menu-driven C++ console application that supports the following operations:

- Add a student with a unique ID, name, and GPA (validated to the range 0.0–4.0).
- Remove a student from the registry by their ID.
- Search for a student by ID and display their profile.
- Display all registered students in a formatted table ordered by ID.
- Enroll a student in a course, with automatic duplicate-course prevention.
- Display the full course list for any student.
- Sort all students by GPA (descending).
- Exit the program cleanly.

2.1 Object-Oriented Programming

The program uses a two-class hierarchy. Person is a base class with a virtual display() method. Student inherits from Person and overrides display(), implementing Encapsulation, Inheritance, and Polymorphism.

2.2 STL Containers

Container	Location	Purpose
map<int, Student>	CourseManager	Primary registry — $O(\log n)$ lookup by student ID
set<string>	Student class	Per-student course list — auto-deduplication and sorted order
vector<Student*>	sortByGpa()	Temporary pointer list for GPA sorting without copying objects

2.3 STL Algorithms

- sort() — sorts vector<Student*> by GPA descending using a lambda comparator.

3. Project Beneficiaries

- Academic Administrators — a fast, reliable tool for managing student records without manual spreadsheets.
- Instructors can quickly search students, view enrollments, and identify high-achievers by GPA ranking.
- Students (indirectly) — accurate enrollment records prevent duplicate or conflicting course registrations.
- C++ Learners — the codebase is a clean, well-commented example of OOP and STL usage in a real application.

4. Detailed Analysis

4.1 Problem Statement

Manual or spreadsheet-based student management lacks programmatic validation, duplicate prevention, and fast lookup. The SCMS addresses these limitations through structured data models and algorithms.

4.2 Functional Requirements

ID	Requirement	Implementation
FR-01	Unique student ID	map key uniqueness check via <code>students.count()</code>
FR-02	GPA 0.0–4.0 validation	<code>if (gpa < 0 gpa > 4)</code> guard in <code>run()</code>
FR-03	No duplicate courses	<code>set<string></code> auto-rejects duplicate insertions
FR-04	Graceful invalid input	<code>cin.ignore()</code> after numeric reads
FR-05	Sort students by GPA	STL <code>sort()</code> on <code>vector<Student*></code> with lambda

4.3 Design Decisions

- `map<int, Student>` was chosen over `unordered_map` to maintain key-sorted order, enabling ordered display by ID without extra sorting.
- `set<string>` was chosen for course storage because it eliminates duplicates automatically and keeps courses alphabetically sorted.
- `vector<Student*>` was used for GPA sort to avoid copying Student objects, improving memory efficiency.
- The base class `Person` provides a virtual `display()` method that `Student` overrides, demonstrating polymorphism.

5. Task Breakdown Structure

Task	Description	Hours
T1 – Requirements Analysis	Read assignment spec; identify all functional and constraint requirements.	2h
T2 – OOP Design	Design class hierarchy; define attributes, methods, and a UML diagram.	3h
T3 – STL Container Selection	Evaluate map, set, vector; justify choices based on access patterns.	1h
T4 – Core Implementation	Implement Person, Student, CourseManager, all menu functions, and STL logic.	8h
T5 – Input Validation	Implement cin.ignore() and range checks for all user inputs.	2h
T6 – Testing	Define and run all 12 test cases; verify edge cases.	3h
T7 – Report Writing	Document all phases, write sections, produce README.	4h

6. Time Plan — Gantt Chart

The project was completed in one week. The table below shows the day-by-day task allocation:

Task	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
T1 – Requirements	■						
T2 – OOP Design		■	■				
T3 – STL Selection		■					
T4 – Implementation			■	■	■		
T5 – Validation				■	■		
T6 – Testing					■	■	
T7 – Report						■	■

Shaded cells (■) indicate active work on that day.

7. System Architecture and Design

7.1 UML Class Diagram

The system is composed of three components: the base class Person, the derived class Student, and the CourseManager class. The relationships are described below.

Person
name : string
+ Person(name : string)
+ getName() : string
+ setName(n : string) : void
+ display() : void {virtual}

▲ inherits

Student (inherits Person)
- id : int
- gpa : double
- courses : set<string>
+ Student(id, name, gpa)
+ getId() : int
+ getGpa() : double
+ addCourse(c : string) : bool
+ showCourses() : void
+ display() : void [overrides Person]

CourseManager
- students : map<int, Student>
+ addStudent(s : Student) : void
+ removeStudent(id : int) : void
+ searchStudent(id : int) : Student*
+ displayAll() : void
+ enrollInCourse(id : int, c : string) : void
+ showCourses(id : int) : void
+ sortByGpa() : void
+ run() : void

7.2 File Structure

File	Responsibility
Student.h	Person base class + Student derived class (both in one header)
CourseManager.h	CourseManager class with all management logic and menu
main.cpp	Entry point — instantiates CourseManager and calls run()

7.3 OOP Principles

- Encapsulation: id, gpa, and courses are private in Student; accessed only through public getters.
- Inheritance: Student : public Person — reuses the name attribute and overrides display().
- Polymorphism: the virtual display() in Person allows each subclass to define its own output format.

8. Testing Scenarios and Results

TC#	Test Case	Input	Expected Result	Result
TC-01	Add valid student	ID=1001, GPA=3.5	Added successfully	PASS
TC-02	Duplicate student ID	ID=1001 again	Error: already exists	PASS
TC-03	GPA out of range	GPA=5.0	Rejected with message	PASS
TC-04	Remove existing student	ID=1001	Removed successfully	PASS
TC-05	Remove unknown ID	ID=9999	Error: not found	PASS
TC-06	Search existing student	ID=1001	Full profile shown	PASS
TC-07	Search unknown ID	ID=9999	Error: not found	PASS
TC-08	Enroll in course	Course=Data Structures	Enrolled successfully	PASS
TC-09	Duplicate course enrollment	Same course again	Error: already enrolled	PASS
TC-10	Sort by GPA	3 students added	Ranked descending	PASS
TC-11	Invalid menu input	Choice=abc	Re-prompt, no crash	PASS
TC-12	Empty student list sort	No students	Empty message shown	PASS

8.1 Sample Output

```
Choice: 1
Enter ID: 1001
Enter Name: Ali Ahmed
Enter GPA: 3.5
Student added successfully
Choice: 7
Rank   ID      Name           GPA
-----
1      1002    Sara Hassan    3.90
2      1001    Ali Ahmed      3.50
```

9. End-User Guide

9.1 Compile

- Linux / macOS: `g++ -std=c++17 -Wall -o scms main.cpp`
- Windows: `g++ -std=c++17 -Wall -o scms.exe main.cpp`

9.2 Run

- Linux / macOS: `./scms`
- Windows: `scms.exe`

9.3 Menu Options

- Add Student — enter unique ID, full name, and GPA (0.0–4.0).
- Remove Student — enter student ID; system confirms deletion.
- Search Student — enter ID; displays full profile.
- Display All — prints a formatted table of all students ordered by ID.
- Enroll in Course — enter student ID and course name; duplicates are rejected automatically.
- Show Courses — lists all enrolled courses for a given student.
- Sort by GPA — prints students ranked by GPA descending.
- Exit — terminates the program cleanly.

9.4 Input Rules

- Student IDs must be unique positive integers.
- GPA must be a decimal number between 0.0 and 4.0 (inclusive).
- Names and course names may include spaces (getline is used after `cin.ignore()`).
- Non-numeric GPA input is validated and rejected with an error message.

10. Conclusion

The Student Course Management System successfully meets all eight functional requirements of the assignment. The system demonstrates a clean application of C++ OOP principles through a two-class hierarchy (Person → Student), uses three distinct STL containers (map, set, vector), and applies the STL sort() algorithm with a lambda expression for GPA ranking.

All 12 test cases passed, including edge cases for duplicate IDs, out-of-range GPA values, duplicate course enrollment, and invalid user input. The two-header file structure (Student.h, CourseManager.h, main.cpp) reflects clear separation of concerns.

Potential future enhancements include file-based data persistence using fstream, a count_if() query for high-achiever statistics (GPA >= 3.0), a database backend for larger datasets, and a graphical user interface. The modular architecture of the current design would support these extensions with minimal refactoring.

11. References

- [1] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- [2] Josuttis, N. M. (2012). The C++ Standard Library: A Tutorial and Reference (2nd ed.). Addison-Wesley.
- [3] cppreference.com. (2024). C++ Standard Library – Containers.
<https://en.cppreference.com/w/cpp/container>
- [4] cppreference.com. (2024). C++ Standard Library – Algorithms.
<https://en.cppreference.com/w/cpp/algorithm>
- [5] ISO/IEC 14882:2017. Programming Languages – C++. International Organization for Standardization.
- [6] GitHub Repository: <https://github.com/iMohammedJameel/Student-Course-Management-System>